

Servlets - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SERVLETS — MASTER NOTES	4
A Complete Reference Book for Interviews, Production Work & System Understanding	4
TOPIC 1: SERVLET LIFECYCLE	4
1. Introduction	4
2. The Five Lifecycle Phases	4
3. Diagram — Full Lifecycle Timeline	5
4. Real Life Analogy	5
5. Critical Thread-Safety Implication	5
6. Edge Cases	6
7. Common Mistakes	6
8. Best Practices	6
9. Frequently Asked Interview Questions	6
TOPIC 2: GENERICSERVLET	7
1. Introduction	7
2. Why GenericServlet Exists (and Why You Rarely Extend It Directly)	7
3. Code Example	7
4. Frequently Asked Interview Questions	7
TOPIC 3: HTTPSERVLET	8
1. Introduction	8
2. Key Methods to Override	8
3. Internal Working — How service() Dispatches	8
4. Code Example	9
5. Edge Cases	9
6. Common Mistakes	9
7. Frequently Asked Interview Questions	9
TOPIC 4: REQUEST & RESPONSE (HttpServletRequest & HttpServletResponse)	

1. Introduction	10
2. Key HttpServletRequest Methods	10
3. Key HttpServletResponse Methods	10
4. sendRedirect() vs RequestDispatcher.forward() — A Critical Distinction	11
5. Code Example	11
6. Edge Cases	11
7. Common Mistakes	11
8. Frequently Asked Interview Questions	12
TOPIC 5: SESSION MANAGEMENT (CROSS-REFERENCE)	12
TOPIC 6: COOKIES (CROSS-REFERENCE)	12
TOPIC 7: FILTERS (CROSS-REFERENCE)	12
TOPIC 8: LISTENERS (CROSS-REFERENCE)	13
TOPIC 9: SERVLETCONFIG	13
1. Introduction	13
2. Code Example	13
3. web.xml Equivalent	13
4. Frequently Asked Interview Questions	13
TOPIC 10: SERVLETCONTEXT	14
1. Introduction	14
2. ServletConfig vs ServletContext — Comparison Table	14
3. Code Example	14
4. Key Methods	15
5. The Three Attribute Scopes — Full Comparison (Consolidating Request/Session/Context)	15
6. Frequently Asked Interview Questions	15
7. Revision Notes (Topics 1-4, 9-10 Consolidated)	16
8. Homework (Covers Whole Servlets Section)	16
FINAL SUMMARY — SERVLETS COMPLETE	16

SERVLETS — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: SERVLET LIFECYCLE

1. Introduction

What is it? A Servlet is a Java class that handles HTTP requests within a Servlet container (Tomcat, Jetty, Undertow) — the foundational technology underneath JSP, and, indirectly, Spring MVC itself. The Servlet lifecycle defines the exact, container-managed sequence of phases every Servlet instance goes through: loading, initialization, request servicing, and destruction.

Why it matters: Understanding this lifecycle explains WHY Servlets are efficient (one instance serves MANY requests, not one instance per request) and reveals thread-safety implications that trip up many developers (a single Servlet instance is shared across CONCURRENT requests from different threads).

2. The Five Lifecycle Phases

1. **LOADING** -> container loads the Servlet's .class file (usually lazily, on first request, UNLESS load-on-startup is configured)
2. **INSTANTIATION** -> container creates exactly ONE instance of the Servlet class (via its no-arg constructor) - NOT one instance per request!
3. **init()** -> called EXACTLY ONCE, right after instantiation, before handling any requests (one-time setup: DB connections, config loading)
4. **service()** [repeated] -> called ONCE PER INCOMING REQUEST, for the ENTIRE lifetime of the Servlet - dispatches internally to doGet()/doPost()/etc. based on HTTP method
5. **destroy()** -> called EXACTLY ONCE, when the container shuts down or the Servlet is being removed from service (cleanup: closing resources)

3. Diagram — Full Lifecycle Timeline

```
Container Startup / First Request
|
v
[Class Loaded]
|
v
[new Servlet()] <- instantiated ONCE
|
v
init() <- ONCE, before any request handling
|
v
+-----+
| service() loop | <- called for EVERY incoming request,
| (many, many times) | for the ENTIRE life of this instance
+-----+
|
v (container shutdown, or Servlet removed)
destroy() <- ONCE, final cleanup
|
v
[Instance eligible for GC]
```

4. Real Life Analogy

- **Restaurant:** The Servlet lifecycle is like hiring ONE specific chef (Servlet instance) who sets up their station ONCE at the start of their shift (`init()`), then cooks MANY different dishes (`service()` calls) for MANY different customers throughout the entire shift using that same station, and finally cleans up and packs away their station ONCE at the end of the shift (`destroy()`) — the chef is NOT re-hired and re-trained for every single dish ordered.

5. Critical Thread-Safety Implication

Because ONE Servlet instance handles MANY CONCURRENT requests (each on its OWN thread, managed by the container's thread pool), **instance fields on a Servlet are effectively SHARED, mutable state across all concurrently executing threads** — exactly the race-condition risk covered in [\[\[Synchronization\]\]](#) (Core Java, Multithreading). Storing per-request mutable state as an instance field is a serious, classic Servlet bug.

```
public class BadCounterServlet extends HttpServlet {
    private int requestCount = 0; // DANGEROUS - shared across ALL concurrent requests!

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        requestCount++; // RACE CONDITION - not synchronized, not per-request-safe
        // ...
    }
}
```

Correct approach: Use LOCAL variables inside `service()/doGet()/etc.` (each thread gets its own Stack frame — inherently safe), or genuinely shared state managed with proper synchronization / atomic types (`AtomicInteger`) if a true shared counter is intended.

6. Edge Cases

Case	Result
An exception thrown inside <code>init()</code>	The Servlet fails to initialize — the container will typically not route requests to it, may retry initialization on a later request depending on configuration
Multiple concurrent requests to the SAME Servlet	All share the SAME instance — each runs <code>service()</code> on its OWN thread, but any instance fields are genuinely shared
<code>destroy()</code> called while requests are still actively in <code>service()</code>	The container generally attempts to let in-flight requests complete before or during shutdown, per the Servlet Specification's graceful shutdown expectations

7. Common Mistakes

- Storing per-request or per-user mutable state as Servlet INSTANCE fields — a classic, serious concurrency bug, since one instance is shared across all concurrent requests.
- Assuming a NEW Servlet instance is created for every request — it is NOT; exactly one instance (by default) serves the Servlet's entire operational lifetime.
- Performing expensive, per-request setup logic inside `service()` that could instead be done ONCE inside `init()`.

8. Best Practices

- Keep Servlets STATELESS with respect to per-request data — use only local variables for anything request-specific.
- Use `init()` for genuine one-time setup (reading configuration, establishing shared resources).
- If genuinely shared mutable state is required, protect it properly (synchronization, atomic types, or better, delegate to a properly-designed thread-safe service class).

9. Frequently Asked Interview Questions

- 1 **How many instances of a Servlet does a container typically create?** Exactly one, by default — that single instance handles ALL incoming requests for the Servlet's entire lifetime.
- 2 **What are the five phases of the Servlet lifecycle?** Loading, Instantiation, `init()` (once), `service()` (repeated per request), `destroy()` (once).
- 3 **Why is storing per-request state in a Servlet instance field dangerous?** Because the SAME instance handles many concurrent requests on different threads — instance fields are shared mutable state, creating race conditions exactly like any other multithreaded shared-state bug.
- 4 **When is `init()` called, and how many times?** Exactly once, immediately after the Servlet is instantiated, before it handles its first request.
- 5 **What's the safe alternative to storing per-request data as an instance field?** Use local variables declared inside `service()/doGet()/doPost()` — each thread gets its own independent copy on its own Stack frame.

TOPIC 2: GENERICSERVLET

1. Introduction

What is it? `GenericServlet` is an abstract class implementing the `Servlet` interface, providing a protocol-independent (not HTTP-specific) base implementation — handling boilerplate like storing the `ServletConfig` and implementing most `Servlet` interface methods, leaving only `service()` abstract for subclasses to implement.

2. Why `GenericServlet` Exists (and Why You Rarely Extend It Directly)

`GenericServlet` is deliberately protocol-agnostic — it doesn't know about HTTP specifically (no `doGet/doPost` concepts). In practice, virtually ALL real-world Servlets are HTTP-based, so developers extend `HttpServlet` (next topic) instead, which itself extends `GenericServlet` and adds the HTTP-specific method dispatching.

```
Servlet (interface)
|
GenericServlet (abstract, protocol-agnostic)
|
HttpServlet (abstract, HTTP-specific - THIS is what you actually extend in practice)
|
YourServlet (your concrete implementation)
```

3. Code Example

```
import jakarta.servlet.*;
import java.io.IOException;

public class MyGenericServlet extends GenericServlet {
    @Override
    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/plain");
        response.getWriter().println("Handled by a GenericServlet");
    }
}
```

4. Frequently Asked Interview Questions

- 1 **What does `GenericServlet` provide?** A protocol-independent base implementation of the `Servlet` interface, handling most boilerplate and leaving `service()` abstract.
- 2 **Why do developers almost always extend `HttpServlet` instead of `GenericServlet` directly?** Because virtually all real Servlets handle HTTP specifically, and `HttpServlet` provides convenient HTTP-method-specific dispatching (`doGet`, `doPost`, etc.) that `GenericServlet` does not.
- 3 **What's the class hierarchy relationship between `GenericServlet` and `HttpServlet`?** `HttpServlet` extends `GenericServlet`, adding HTTP-specific method dispatch on top of its protocol-agnostic base.

TOPIC 3: HTTPSERVLET

1. Introduction

What is it? `HttpServlet` is the abstract class virtually every real-world Servlet extends — it implements `GenericServlet`'s `service()` method to automatically dispatch incoming requests to the appropriate `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc., based on the actual HTTP method used.

2. Key Methods to Override

Method	HTTP Method Handled
<code>doGet(request, response)</code>	GET
<code>doPost(request, response)</code>	POST
<code>doPut(request, response)</code>	PUT
<code>doDelete(request, response)</code>	DELETE
<code>doHead(request, response)</code>	HEAD (default implementation calls <code>doGet()</code> and discards the body, per HTTP spec)
<code>init()/destroy()</code>	Lifecycle hooks (inherited, same as <code>GenericServlet</code>)

3. Internal Working — How `service()` Dispatches

```
// Conceptually, HttpServlet's service() method does something like:
protected void service(HttpServletRequest req, HttpServletResponse resp) {
    String method = req.getMethod();
    switch (method) {
        case "GET" -> doGet(req, resp);
        case "POST" -> doPost(req, resp);
        case "PUT" -> doPut(req, resp);
        // ... etc.
        default -> /* unsupported method handling */;
    }
}
```

You NEVER override `service()` directly when extending `HttpServlet` — you override the SPECIFIC `doxxx()` method(s) corresponding to the HTTP methods you actually want to handle.

4. Code Example

```
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;
import java.io.*;

@WebServlet("/greet")
public class GreetServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        String name = request.getParameter("name");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>Hello, " + (name != null ? name : "Guest") + "!</h1>");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        response.getWriter().println("Received a POST request");
    }
}
```

5. Edge Cases

Case	Result
A request arrives with an HTTP method NOT overridden (e.g., DELETE sent, only doGet overridden)	The default <code>HttpServlet</code> implementation responds with <code>405 Method Not Allowed</code>
<code>doGet()</code> overridden, but <code>doHead()</code> not	<code>doHead()</code> 's default implementation automatically calls <code>doGet()</code> internally and strips the response body, per the HTTP spec's HEAD semantics

6. Common Mistakes

- Overriding `service()` directly instead of the specific `doXxx()` methods — loses the convenient HTTP-method-based dispatch `HttpServlet` provides, and is essentially never the right approach.
- Forgetting that unhandled HTTP methods automatically return `405`, and being confused why a request "does nothing" when actually the WRONG `doXxx()` method was implemented for the client's actual request method.

7. Frequently Asked Interview Questions

- 1 **What does `HttpServlet` add over `GenericServlet`?** HTTP-method-specific request dispatching — automatically routing to `doGet()`, `doPost()`, etc., based on the incoming request's HTTP method.
- 2 **Which method should you override when extending `HttpServlet`?** The specific `doXxx()` method(s) matching the HTTP methods you want to handle (`doGet`, `doPost`, etc.) — NOT `service()` directly.
- 3 **What happens if a client sends a request using an HTTP method your Servlet doesn't override?** `HttpServlet`'s default behavior responds with `405 Method Not Allowed`.

TOPIC 4: REQUEST & RESPONSE

(HttpServletRequest & HttpServletResponse)

1. Introduction

What is it? `HttpServletRequest` and `HttpServletResponse` are the two objects the container passes into every `doGet()`/`doPost()` call, representing the incoming HTTP request and the outgoing HTTP response respectively — the primary way a Servlet reads client input and produces output.

2. Key `HttpServletRequest` Methods

Method	Purpose
<code>getParameter(name)</code>	Retrieves a single query-string or form parameter value
<code>getParameterValues(name)</code>	Retrieves ALL values for a parameter that may appear multiple times (e.g., checkboxes)
<code>getHeader(name)</code>	Retrieves an HTTP request header value
<code>getMethod()</code>	The HTTP method used (GET, POST, etc.)
<code>getRequestURI()</code>	The requested URL path
<code>getSession()</code>	Retrieves (or creates) the associated <code>HttpSession</code> (see [[Session Management]] , Advanced Java Topic 4)
<code>getInputStream()</code> / <code>getReader()</code>	Reads the raw REQUEST BODY (e.g., for JSON payloads in a REST-style Servlet)
<code>setAttribute(name, value)</code> / <code>getAttribute(name)</code>	Request-SCOPED data, commonly used to pass data forward in an MVC-style forward (see [[MVC Architecture]] , Advanced Java Topic 2)

3. Key `HttpServletResponse` Methods

Method	Purpose
<code>setContentType(type)</code>	Sets the response's Content-Type header (e.g., "text/html", "application/json")
<code>getWriter()</code>	Returns a <code>PrintWriter</code> for writing TEXT-based response content
<code>getOutputStream()</code>	Returns a <code>ServletOutputStream</code> for writing BINARY response content
<code>setStatus(code)</code>	Sets the HTTP response status code (200, 404, 500, etc.)
<code>sendRedirect(url)</code>	Sends a 302 redirect, instructing the browser to make a NEW request to a different URL
<code>addCookie(cookie)</code>	Adds a Set-Cookie header (see [[Cookies]] , Advanced Java Topic 5)
<code>addHeader(name, value)</code>	Adds an arbitrary response header

4. `sendRedirect()` VS `RequestDispatcher.forward()` — A Critical Distinction

Aspect	<code>response.sendRedirect(url)</code>	<code>request.getRequestDispatcher(url).forward(req, resp)</code>
Round trip	Browser makes a BRAND-NEW request to the new URL (2 full HTTP round-trips total)	Happens entirely SERVER-SIDE — no new browser request at all
URL bar changes?	Yes — browser's URL updates to the new location	No — browser's URL bar stays the same
Request data preserved?	No — it's a genuinely new request, original request attributes are lost	Yes — the SAME <code>HttpServletRequest/HttpServletResponse</code> objects (and any <code>setAttribute()</code> data) are passed along
Typical use	Redirecting to an entirely different resource/site after processing (e.g., post-login redirect)	Internal MVC-style forwarding to a View (JSP) with Model data attached

5. Code Example

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (isValid(username, password)) {
            request.getSession().setAttribute("user", username);
            response.sendRedirect("dashboard.jsp"); // NEW request, URL bar changes
        } else {
            request.setAttribute("error", "Invalid credentials");
            request.getRequestDispatcher("login.jsp").forward(request, response); // SAME request, forwards data
        }
    }

    private boolean isValid(String u, String p) { return "admin".equals(u) && "secret".equals(p); }
}
```

6. Edge Cases

Case	Result
Calling <code>getParameter()</code> on a name that doesn't exist in the request	Returns <code>null</code>
Calling <code>getWriter()</code> AND <code>getOutputStream()</code> on the SAME response	<code>IllegalStateException</code> — only ONE of the two can be used per response
Calling <code>sendRedirect()</code> AFTER the response has already been committed (partially written)	<code>IllegalStateException</code>

7. Common Mistakes

- Confusing `sendRedirect()` (new browser request, URL changes) with `forward()` (server-side only, URL unchanged) — using the wrong one breaks the intended user experience or loses request-scoped data.

- Calling both `getWriter()` and `getOutputStream()` on the same response object.
- Forgetting `getParameter()` returns `null` (not an empty string or exception) for a missing parameter, causing `NullPointerException` if used without a null-check.

8. Frequently Asked Interview Questions

- 1 **What's the difference between `sendRedirect()` and `forward()`?** `sendRedirect()` tells the BROWSER to make a brand-new request to a different URL (URL bar changes, original request data is lost); `forward()` happens entirely server-side, passing the SAME request/response objects along (URL bar unchanged, request attributes preserved).
 - 2 **What does `request.getParameter("name")` return if the parameter doesn't exist?** `null`.
 - 3 **Can you call both `getWriter()` and `getOutputStream()` on the same `HttpServletResponse`?** No — doing so throws `IllegalStateException`; only one can be used per response.
 - 4 **How do you pass data from a Servlet to a JSP View using `forward()`?** Use `request.setAttribute(name, value)` before forwarding — the JSP can then read that same attribute from the shared request object.
-

TOPIC 5: SESSION MANAGEMENT (CROSS-REFERENCE)

(Fully covered in Advanced Java, Topic 4 — `HttpSession`, the `JSESSIONID` mechanism, and load-balanced session-sharing challenges apply identically in the Servlet context, since `HttpSession` IS a Servlet API construct. No new content needed here — see that topic for full depth.)

TOPIC 6: COOKIES (CROSS-REFERENCE)

(Fully covered in Advanced Java, Topic 5 — cookie creation/reading via `HttpServletRequest.getCookies()`/`HttpServletResponse.addCookie()`, and security attributes like `HttpOnly`/`Secure`/`SameSite`, apply identically here.)

TOPIC 7: FILTERS (CROSS-REFERENCE)

(Fully covered in Advanced Java, Topic 6 — the `Filter` interface, the filter chain, and `chain.doFilter()` are Servlet API constructs used identically in this context.)

TOPIC 8: LISTENERS (CROSS-REFERENCE)

(Fully covered in Advanced Java, Topic 7 — `ServletContextListener`, `HttpSessionListener`, and related lifecycle-event listeners are Servlet API constructs used identically here.)

TOPIC 9: SERVLETCONFIG

1. Introduction

What is it? `ServletConfig` holds configuration information SPECIFIC TO ONE SERVLET — primarily its initialization parameters, set either via `web.xml` or the `@WebServlet` annotation's `initParams` attribute — accessible from within that Servlet via `getServletConfig()`.

2. Code Example

```
@WebServlet(urlPatterns = "/report",
initParams = { @WebInitParam(name = "maxRows", value = "100") })
public class ReportServlet extends HttpServlet {

    private int maxRows;

    @Override
    public void init() throws ServletException {
        String value = getServletConfig().getInitParameter("maxRows");
        maxRows = Integer.parseInt(value);
    }

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        resp.getWriter().println("Max rows configured: " + maxRows);
    }
}
```

3. `web.xml` Equivalent

```
<servlet>
<servlet-name>ReportServlet</servlet-name>
<servlet-class>com.company.ReportServlet</servlet-class>
<init-param>
<param-name>maxRows</param-name>
<param-value>100</param-value>
</init-param>
</servlet>
```

4. Frequently Asked Interview Questions

- 1 **What does `ServletConfig` hold?** Initialization parameters specific to ONE particular Servlet.
- 2 **When would you typically read `ServletConfig` values?** Inside `init()`, since it's called once, before the Servlet begins handling requests — an ideal place for one-time configuration loading.

- 3 **How is `ServletConfig` scoped compared to `ServletContext` (next topic)?** `ServletConfig` is scoped to ONE Servlet; `ServletContext` is scoped to the ENTIRE web application.
-

TOPIC 10: SERVLETCONTEXT

1. Introduction

What is it? `ServletContext` represents the ENTIRE web application as a whole (shared across ALL Servlets, Filters, and JSPs within that application) — used for application-wide configuration parameters, inter-Servlet communication via shared attributes, and accessing application resources (files bundled in the WAR).

2. `ServletConfig` VS `ServletContext` — Comparison Table

Aspect	<code>ServletConfig</code>	<code>ServletContext</code>
Scope	ONE specific Servlet	The ENTIRE web application
Typical use	Servlet-specific initialization parameters	Application-wide configuration, shared data across Servlets, resource access
Obtained via	<code>getServletConfig()</code>	<code>getServletContext()</code> (available from BOTH <code>ServletConfig</code> and directly on any Servlet/Filter)

3. Code Example

```
@WebServlet("/visitCounter")
public class VisitCounterServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        ServletContext context = getServletContext();

        Integer count = (Integer) context.getAttribute("totalVisits");
        if (count == null) count = 0;
        count++;
        context.setAttribute("totalVisits", count); // shared, application-wide state - ALL users/Servlets see this

        resp.getWriter().println("Total application visits: " + count);
    }
}
```

Unlike `HttpSession` (per-USER state) or request attributes (per-REQUEST state), `ServletContext` attributes are shared across the ENTIRE APPLICATION and EVERY user — meaning the SAME thread-safety concerns from Topic 1 apply here too, and often MORE acutely, since `ServletContext` state is shared across truly ALL concurrent users, not just concurrent requests to one Servlet.

4. Key Methods

Method	Purpose
<code>getInitParameter(name)</code>	Application-wide (not Servlet-specific) initialization parameters, configured in <code>web.xml</code> 's <code><context-param></code>
<code>setAttribute(name, value) / getAttribute(name)</code>	Application-wide shared data
<code>getRealPath(path)</code>	Resolves a resource's real filesystem path within the deployed application
<code>log(message)</code>	Writes to the container's log

5. The Three Attribute Scopes — Full Comparison (Consolidating Request/Session/Context)

Scope	Object	Lifetime	Shared Across
Request	<code>HttpServletRequest</code>	One single request	Nothing — unique per request
Session	<code>HttpSession</code>	One user's session (until timeout/invalidate)	All requests from the SAME user
Application	<code>ServletContext</code>	The entire application's lifetime	ALL users, ALL requests, ALL Servlets

6. Frequently Asked Interview Questions

- 1 **What does `ServletContext` represent?** The entire web application, shared across all Servlets, Filters, and JSPs within it.
- 2 **What's the key difference between `ServletConfig` and `ServletContext`?** `ServletConfig` is scoped to ONE Servlet's own initialization parameters; `ServletContext` is scoped to the ENTIRE application, shared by everything within it.
- 3 **What are the three attribute scopes in a Servlet application, from narrowest to widest?** Request scope (one request), Session scope (one user across requests), Application/Context scope (the entire application, all users).
- 4 **Why does storing mutable data in `ServletContext` attributes require even more care than Servlet instance fields?** Because it's shared across EVERY concurrent user and request in the entire application, not just concurrent requests to one specific Servlet — the same race-condition risks apply, at an even broader scale.

7. Revision Notes (Topics 1-4, 9-10 Consolidated)

SERVLETS — CHEAT SHEET

=====

LIFECYCLE: Load -> Instantiate (ONCE) -> init() (ONCE) -> service() (per request, MANY times) -> destroy() (ONCE)

GOTCHA: instance fields are SHARED across all concurrent requests - race condition risk!

Servlet (interface) -> GenericServlet (protocol-agnostic) -> HttpServlet (HTTP-specific, what you actually extend) -> override doGet()/doPost()/etc., NEVER service() directly

REQUEST: getParameter(), getSession(), setAttribute()/getAttribute() (request scope)

RESPONSE: setContentType(), getWriter()/getOutputStream() (only ONE per response), sendRedirect() (NEW browser request, URL changes) vs forward() (server-side, URL unchanged, request data preserved)

ServletConfig -> ONE Servlet's init params

ServletContext -> WHOLE APPLICATION's shared config/data (all users, all Servlets)

THREE ATTRIBUTE SCOPES: Request < Session < Application (ServletContext)

8. Homework (Covers Whole Servlets Section)

- 1 Build a simple `HttpServlet` with `doGet()` reading a query parameter and `doPost()` reading form data, and test both with a browser/curl.
- 2 Demonstrate the `sendRedirect()` vs `forward()` difference by building two small Servlets, observing the browser's URL bar and whether `request.setAttribute()` data survives in each case.
- 3 Build a `ServletContext`-based visit counter (as shown in Topic 10) and verify it increments correctly across multiple browser tabs/sessions (since it's application-scoped, not session-scoped).

FINAL SUMMARY — SERVLETS COMPLETE

This concludes the **Servlets** section — 10 topics (4 fully new + 4 cross-referenced to Advanced Java + 2 new):

- 1 **Servlet Lifecycle** — Loading, Instantiation (once), `init()` (once), `service()` (per request), `destroy()` (once); the critical thread-safety implication of shared instance fields.
- 2 **GenericServlet** — the protocol-agnostic base class, why you extend `HttpServlet` instead.
- 3 **HttpServlet** — HTTP-method-specific `doGet()/doPost()/etc.` dispatching.
- 4 **Request & Response** — `HttpServletRequest/HttpServletResponse` core methods, `sendRedirect()` vs `forward()`.
5-8. **Session Management, Cookies, Filters, Listeners** — cross-referenced to Advanced Java Topics 4-7 (identical concepts, no duplication needed).
- 5 **ServletConfig** — per-Servlet initialization parameters.
- 6 **ServletContext** — application-wide shared configuration/state, the three attribute scopes.

This is the exact foundational layer that JSP (next section) builds its templating on top of, and that Spring MVC abstracts away almost entirely — you now understand precisely what's happening beneath both.

(END OF SERVLETS MASTER NOTES — ready to proceed to JSP whenever you say "Next Topic.")